

LESSON 7 SVD, Data Compression, PCA, Dimension Reduction

7.0 Review: SVD and Data Compression

In MATH2331, we learn that every $m \times n$ matrix A of rank r has a singular value decomposition, that is, $A = U\Sigma V^T$, where U and V are orthogonal, and Σ is diagonal with non-negative singular values in decreasing order: $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > \sigma_{r+1} = \sigma_m = 0$. Unless $r = n = m$, there are zeros on the diagonals of Σ that do not contribute to matrix product $U\Sigma V^T$. We may reduce $U\Sigma V^T$ to $U_r \Sigma_r V_r^T$.

$$A = \begin{bmatrix} | & & | \\ u_1 & \cdots & u_m \\ | & & | \end{bmatrix} \begin{bmatrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_r \\ & & & 0 \end{bmatrix} \begin{bmatrix} - & v_1^T & - \\ & \vdots & \\ - & v_n^T & - \end{bmatrix} = \begin{bmatrix} | & & | \\ u_1 & \cdots & u_r \\ | & & | \end{bmatrix} \begin{bmatrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_r \end{bmatrix} \begin{bmatrix} - & v_1^T & - \\ & \vdots & \\ - & v_r^T & - \end{bmatrix}$$

$$\begin{matrix} A = & U & \Sigma & V^T & U_r & \Sigma_r & V_r^T \\ m \times n & m \times m & m \times n & n \times n & m \times r & r \times r & r \times n \end{matrix}$$

SVD leads to an important sum of **rank 1 matrices**:

$$A = \sigma_1 u_1 v_1^T + \dots + \sigma_r u_r v_r^T$$

where the terms contribute to A in decreasing order since $\sigma_1 \geq \dots \geq \sigma_r$ and the singular vectors u_i 's and v_i 's are unit vectors. Consider the following example of reduced SVD from MATH 2331 notes,

$$A = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ 1/\sqrt{2} & -1/\sqrt{2} \end{bmatrix} \begin{bmatrix} \sqrt{3} & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1/\sqrt{6} & 2/\sqrt{6} & 1/\sqrt{6} \\ -1/\sqrt{2} & 0 & 1/\sqrt{2} \end{bmatrix}$$

$$A = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix} = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T = \sqrt{3} \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \frac{1}{\sqrt{6}} [1 \ 2 \ 1] + \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} \frac{1}{\sqrt{2}} [-1 \ 0 \ 1]$$

$$= \frac{1}{2} \begin{bmatrix} 1 & 2 & 1 \\ 1 & 2 & 1 \end{bmatrix} + \frac{1}{2} \begin{bmatrix} -1 & 0 & 1 \\ 1 & 0 & -1 \end{bmatrix}$$

The best rank-1 approximation of A is $\frac{1}{2} \begin{bmatrix} 1 & 2 & 1 \\ 1 & 2 & 1 \end{bmatrix}$.

One of the applications of SVD is **image compression**. Suppose we want to transmit a picture with 1000×1000 pixels, each of which has a color represented by a number. We need to transmit a 1000×1000 matrix A which contains 10^6 entries. However, A may be well approximated by a small subset of the rank-1 matrices. Consider the painting Wave in 860 $\times$ 1280 pixels. The original picture is on the left. If we use only singular values which are greater than $0.01\sigma_1$, where σ_1 is the largest singular value, then we use the first 120 singular values, and we send 120 vectors of length 860 and 120 vectors of length 1280, i.e. $120 \times (860 + 1280) = 256,800$ numbers, less than 25% of the original load $860 \times 1280 = 1100800$. The quality of the image on the right is still quite good.



As we increase the cutoff, we transmit less (“cheaper”) but we also lose quality (“blurrer”), we will carry this out in homework. The upshot is this:

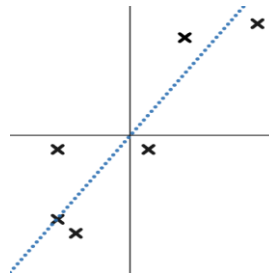
- The *best* rank- k matrix to approximate A is

$$A_k = \sigma_1 u_1 v_1^T + \cdots + \sigma_k u_k v_k^T$$

7.1 Principal Component Analysis (PCA) and Dimension Reduction

We will give another application of SVD, called **Principal Component Analysis (PCA)**. Instead of an image matrix, we have a data matrix X . **Principal components** are simply singular vectors v_i ’s or equivalently eigenvectors of $X^T X$. The idea is the same as image compression, we replace X by the best rank k approximation of X .

We give a simple example. Suppose we have two columns of data, ages and heights, as independent variables for predicting, say, weights. The question is: are the two columns of age and height data truly independent? We *center* the data by subtracting the average/mean age and average height, and plot a versus b :

$$X = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} \\ \vdots & \vdots \\ x_1^{(n)} & x_2^{(n)} \end{bmatrix} \Rightarrow X = \begin{bmatrix} 3 & 7 \\ -4 & -6 \\ 7 & 8 \\ 1 & -1 \\ -4 & -1 \\ -3 & -7 \end{bmatrix}$$


From the plot, the two variables seem to be dependent and we want to replace the two (not-so-independent) columns of data by one column. In other words, we want to replace X by its best rank-1 approximation. So we do SVD of X using a calculator and get the following reduced SVD of X :

-0.4433	-0.3673
0.4274	0.01314
-0.6253	0.3345
0.01585	0.3542
0.182	-0.7018
0.4433	0.3673

 U_r

16.87	0
0	3.921

 Σ_r

-0.5606	-0.8281
0.8281	-0.5606

 V_r^T

We calculate each rank-1 pieces, $\sigma_1 u_1 v_1^T$ and $\sigma_2 u_2 v_2^T$:

4.192431	6.192922
-4.04206	-5.9708
5.913663	8.73547
-0.1499	-0.22143
-1.72123	-2.54255
-4.19243	-6.19292

 $\sigma_1 u_1 v_1^T$
 $+$

-1.19262	0.807367
0.042665	-0.02888
1.086115	-0.73527
1.15008	-0.77857
-2.27873	1.542635
1.192616	-0.80737

 $\sigma_2 u_2 v_2^T$
 $=$

3	7
-4	-6
7	8
1	-1
-4	-1
-3	-7

 X

If we replace A by $\sigma_1 u_1 v_1^T$, since $\sigma_1 u_1 v_1^T$ is a rank 1 matrix, i.e. the second column is just the multiple of the first column, and both are multiple of

$$\sigma_1 u_1 = X v_1 = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 0.5606 \\ 0.8281 \end{bmatrix} = 0.5606x_1 + 0.8281x_2$$

Note that from the calculator, $v_1 = (-0.5606, -0.8281)$, but it could well have given $v_1 = (0.5606, 0.8281)$ and u_1 being negative of what is above. It is better to use the positive eigenvector because then $X v_1$ gives us z_1 which has the same sign as x_1 and x_2 .

We may replace two columns of **highly correlated** data from the original $X = \begin{bmatrix} x_1 & x_2 \end{bmatrix}$ by a single *new* variable z_1 constructed using the first component $v_1 = (0.5606, 0.8281)$:

$$z_1 = \mathbf{0.5606}x_1 + \mathbf{0.8281}x_2$$

thus **reducing the dimension of the data** from 2 to 1. The regression problem with two (not-so-independent) input variables

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

becomes:

$$y = \theta_0 + \theta_1 z_1$$

Note that if we use the negative v_1 , then we will have negative z_1 and negative θ_1 .

We may calculate a second variable using the second component $v_2 = (0.8281, -0.5606)$:

$$z_2 = \mathbf{0.8281}x_1 + (\mathbf{-0.5606})x_2$$

As expected, $y = \theta_0 + \theta_1 x_1 + \theta_2 x_2$ and $y = \theta_0 + \theta_1 z_1 + \theta_2 z_2$ will produce equivalent regression models (homework).

x_1	x_2	$z_1 = 0.5606x_1 + 0.8281x_2$	$z_2 = 0.8281x_1 - 0.5606x_2$
3	7	7.47864	-4.11524
-4	-6	-7.21040	2.72628
7	8	10.54904	-2.70089
1	-1	-0.26740	1.38880
-4	-1	-3.07041	-1.41435
-3	-7	-7.47864	4.11524

7.2 Covariance Matrix

Interestingly, the matrix $X^T X$ (in the SVD of X), or more accurately, $X^T X / (n - 1)$, is the sample **covariance matrix** of the variables x_1 and x_2 , as we now show. Given a sample of size n of a variable x : $x^{(1)}, \dots, x^{(n)}$, the **sample mean** $\bar{x}$ and the **sample variance** s^2 are:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x^{(i)} \quad \text{Var}(x) = s^2 = \frac{1}{n-1} \sum_{i=1}^n (x^{(i)} - \bar{x})^2$$

For a sample of paired variables: $(x_1^{(1)}, x_2^{(1)}), \dots, (x_1^{(n)}, x_2^{(n)})$, we also have the **sample covariance**, which measures how x varies with y :

$$\text{Cov}(x_1, x_2) = \text{Cov}(x_2, x_1) = \frac{1}{n-1} \sum_{i=1}^n (x_1^{(i)} - \bar{x}_1)(x_2^{(i)} - \bar{x}_2)$$

A positive covariance means that y increases when x increases, while a negative covariance means that y decreases when x increases.

For two variables x_1, x_2 , and centered data, we have a 2×2 **covariance matrix**, which contains sample variances and covariances, and turns out to be $X^T X / (n - 1)$:

$$S = \begin{bmatrix} \text{Var}(x_1) & \text{Cov}(x_1, x_2) \\ \text{Cov}(x_2, x_1) & \text{Var}(x_2) \end{bmatrix} = \frac{1}{n-1} \begin{bmatrix} \sum (x_1^{(i)})^2 & \sum x_1^{(i)} x_2^{(i)} \\ \sum x_2^{(i)} x_1^{(i)} & \sum (x_2^{(i)})^2 \end{bmatrix}$$

$$= \frac{1}{n-1} \begin{bmatrix} x_1^{(1)} & \dots & x_1^{(n)} \\ x_2^{(1)} & \dots & x_2^{(n)} \end{bmatrix} \begin{bmatrix} x_1^{(1)} & x_2^{(1)} \\ \vdots & \vdots \\ x_1^{(n)} & x_2^{(n)} \end{bmatrix} = \frac{X^T X}{n-1} = \left(\frac{X}{\sqrt{n-1}} \right)^T \frac{X}{\sqrt{n-1}}$$

7.3 PCA Calculator and Variance Explained

If we use a **PCA calculator**, we get the following eigenvalues and eigenvectors:

$$\begin{aligned} \lambda_1 &= 56.9258, & v_1 &= (0.5606, 0.8281) \\ \lambda_2 &= 3.0742, & v_2 &= (0.8281, -0.5606) \end{aligned}$$

Note that $\sqrt{56.9258} \neq 16.87$, but $\sqrt{5} \sqrt{56.9258} = 16.87 / \sqrt{5}$, because the PCA calculator finds the eigenvalues and eigenvectors of the **covariance matrix** $S = X^T X / (n - 1)$. In other words, PCA of X is the same as SVD of $X / \sqrt{n - 1}$. The eigenvectors and singular vectors are the same in PCA and SVD. There is an additional output from PCA calculator not in the SVD calculator: the **% of variance** explained by each **principal component** v_i ,

$$\text{\% of variance explained by } v_i = \frac{\lambda_i}{\sum \lambda_i}$$

which reflects each component's contribution to *signal* (useful information, as opposed to *noise*). Components with high eigenvalues represent signal, the remaining ones basically reflect noise. We may discard components with low proportion of variance to construct a **low-dimensional model**, this referred to as **dimension reduction**. We can use the square of the singular values from SVD since they are scaled by the same factor $n - 1$. In our example, the percentages for v_1 and v_2 are respectively:

$$\begin{aligned} \frac{56.9258}{56.9258 + 3.0742} &= \frac{16.87^2}{16.87^2 + 3.921^2} = 94.8764\% \\ \frac{3.0742}{56.9258 + 3.0742} &= \frac{3.921^2}{16.87^2 + 3.921^2} = 5.1236\% \end{aligned}$$

The first principal component v_1 captures close to 95% of the data in the sense that even though the data points are on a plane or 2-dimensional space, they are well-approximated by living on a one-dimensional subspace, the blue line. Only the dimension is reduced, the variables are not reduced: each component is a linear combination of the original variables. In a real world problem, we can have very high dimension data, that is, we have a lot of feature variables, which may not be truly independent, PCA helps us sort out the correlation among the original variables and discovers components or **linear combinations** of the original variables to capture useful information.

When we use PCA, we do not just center the variables, but we often also normalize them so that they are on the same scale, one way is to standardize the variables:

$$\frac{x - \mu}{\sigma} \quad \text{approximated by} \quad \frac{x - \bar{x}}{s}$$

where $\bar{x}$ and s are the unbiased sample mean and sample standard deviation. In our example, for x_1 (age) and x_2 (height), we have $\mu_1 = \mu_2 = 0$ (data already centered) and $s_1 = 4.472135955$ and $s_2 = 6.32455532$. If we do PCA on the normalized data, we will get the following principal components (homework).

First component: $[0.70710678 \ 0.70710678]$, variance explained: 94.19417382415922 %

Second component: $[-0.70710678 \ 0.70710678]$, variance explained: 5.805826175840775 %

Normalized x_1	Normalized x_2	$z_1 = 0.7071x_1 + 0.7071x_2$	$z_2 = -0.7071x_1 + 0.7071x_2$
0.670820393	1.106797181	1.25696544	0.30828214
-0.894427191	-0.9486833	-1.30327593	-0.03836486
1.565247584	1.264911064	2.00122437	-0.21236999
0.223606798	-0.15811388	0.04631048	-0.26991728
-0.894427191	-0.15811388	-0.74425893	0.52065213
-0.670820393	-1.10679718	-1.25696544	-0.30828214

We do various regressions in homework:

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

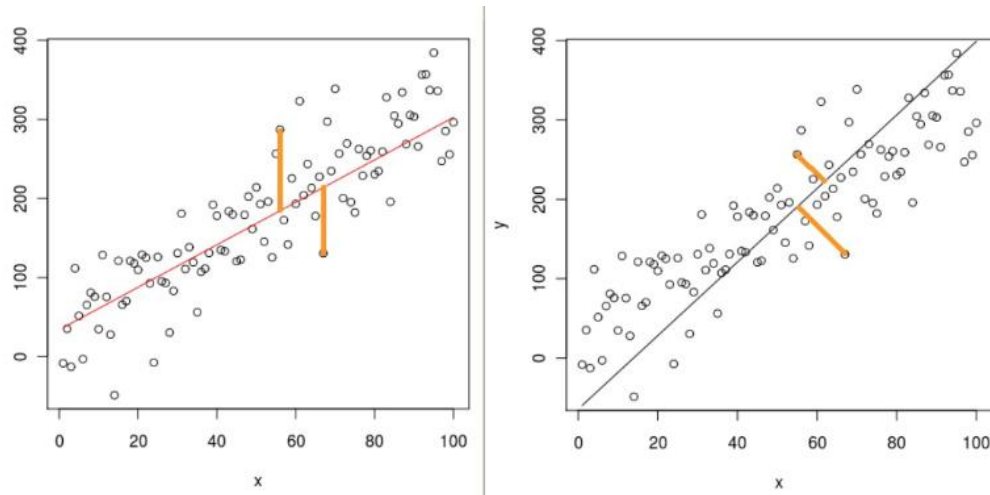
$$y = \theta_0 + \theta_1 z_1$$

$$y = \theta_0 + \theta_1 z_1 + \theta_2 z_2$$

Not surprisingly, $y = \theta_0 + \theta_1 z_1 + \theta_2 z_2$ and $y = \theta_0 + \theta_1 x_1 + \theta_2 x_2$ are the same model, using all the components gives us the full model.

7.4 Best Line: Regression vs PCA

We make a final remark on the *best line*. To find the best line $y = h(x) = \theta_0 + \theta_1 x$ in regression, we minimize the vertical distances $\sum_{i=1}^n (y^{(i)} - h(x^{(i)}))^2$, whereas the best line v_1 in PCA minimizes perpendicular distances. Note that x and y are on the different sides of the regression equation, but the two variables x and y are on the same side in the same matrix.



7.5 Face Recognition

We want to compare an input image to a set of images in a database and find the best match. Each image of $a \times b$ pixels is a row vector of length $n = ab$. For example, the image



$$= \begin{bmatrix} 150 & 152 & \cdot & 151 \\ 131 & 133 & \cdot & 72 \\ \cdot & \cdot & \cdot & \cdot \\ 144 & 171 & \cdot & 67 \end{bmatrix} \quad 128 \times 128$$

is represented by the vector of length $128 \times 128 = 16384$:

$$[150 \ 152 \ \dots \ 151 \ 131 \ 133 \ \dots \ 72 \ \dots \ 144 \ 171 \ \dots \ 67]$$

If we have N images of n pixels each, we can put N row vectors in an $N \times n$ data matrix:

$$\begin{bmatrix} 150 & 152 & \dots & 151 & 131 & 133 & \dots & 72 & \dots & 144 & 171 & \dots & 67 \\ \vdots & & & & & & & & & & & & \\ 131 & 132 & \dots & & & & & & & & & & \end{bmatrix}$$

We expect high correlation among some columns as many adjacent pixels, for example those in the background, may be the same, so we apply PCA on the data matrix. We compute the **mean image** μ , which is an n -vector of the mean of every column, and subtract μ from every row to get a “**centered**” **data matrix** U , and then compute the eigenvectors and eigenvalues of the $n \times n$ **covariance matrix** $U^T U / (N - 1)$. We use m eigenvectors $v_1, v_2, \dots, v_m$ with the m largest eigenvalues; m is usually 20-50 for face recognition, and $m \ll n$. Each image (n -vector) $u^{(i)}$ in the database may be projected onto the span of $v_1, v_2, \dots, v_m$ to result in an m -vector $w^{(i)} = u^{(i)} V_{pca}$, where $V_{pca} = [v_1 \ \dots \ v_m]$. We can project the whole centered data matrix U , and $W = U V_{pca}$ is $N \times m$, where every row is a reduced image, which is still a good representation of the original image, despite massive dimension reduction from $n = 16000$ to $m = 50$. For a new image y to be compared to the images in the database, we center y and project it on the reduced space, $\tilde{y} = (y - \mu) V_{pca}$, and find the closest match among the rows of W to $\tilde{y}$.